# Relative Normal Loss with Lie-Group Probe Curves

An implementation note for Shape-NTFields

October 6, 2026

## 1 Scope

This note covers only the Relative Normal Loss (RNL), Eqs. (6)–(7) of the Shape-NTFields paper. It replaces the rotational probe $r + \varepsilon\, n_{\mathrm{rot}}$, which is a straight line in rotation-vector coordinates, with a curve on the group that rotates the body about its rotation center around the axis $n_{\mathrm{rot}}$.

The network-implied speed $S_c(q; q_g)$ is treated as a black box supplied by the caller. Nothing below depends on how $S_c$ is computed, so this works unchanged if the Eikonal term (and with it the speed definition) has been modified. The one requirement is that RNL use the *same* speed function as the Eikonal term, so that both losses constrain the same quantity.

| Component | Original (paper) | Lie-group version |
|---|---|---|
| Translation probe | $q_s + \varepsilon\,(n_{\mathrm{trans}}, 0)$ | $g\exp(\varepsilon \hat{V}_t)$ — *identical curve* |
| Rotation probe | $q_s + \varepsilon\,(0, n_{\mathrm{rot}})$ | $g\exp(2\pi\varepsilon\, \hat{V}_r)$ — *changes* |
| Speed $S_c$ | caller-defined | unchanged (same function) |
| Slope $G_c$ and penalty (Eq. 7) | finite difference, clipped square | unchanged, plus a cut-locus mask |
| Hyperparameters $\varepsilon, \rho_c, G_{\max}$ | — | carry over (Sec. 5.1) |

Table 1: What the reformulation touches. Only the rotation probe changes in substance.

## 2 Notation

A pose is $g = (R, p) \in \mathrm{SE}(3)$ with $R \in \mathrm{SO}(3)$ and $p \in \mathbb{R}^3$. The network sees $q = (p, r)$ with $r = \mathrm{Log}(R)/(2\pi)$, so $\|r\| \le \frac{1}{2}$. Write $\phi = 2\pi r$ for the rotation vector and $\theta = \|\phi\| \in [0, \pi]$ for the rotation angle.

A twist is $\xi = (\omega, v) \in \mathbb{R}^6 \cong \mathfrak{se}(3)$, with

$$\hat{\xi} = \begin{pmatrix} [\omega]_\times & v \\ 0 & 0 \end{pmatrix}, \qquad [\omega]_\times x = \omega \times x.$$

The curve $t \mapsto g\exp(t\hat{\xi})$ is a *body-frame* motion: expressed in body coordinates, a body point $x$ has velocity $\omega \times x + v$.

The normals $n_{\mathrm{trans}}$ and $n_{\mathrm{rot}}$ are unit vectors in the *world* frame, exactly as in the paper. $n_{\mathrm{trans}}$ points from the nearest body point toward the obstacle. $n_{\mathrm{rot}}$ is the normalized cross product (body radial direction) $\times$ (obstacle radial direction). Let $c \in \mathbb{R}^3$ be the rotation center in body coordinates, and $x_c = Rc + p$ its world position. This must be the same center used to define the radial groups for $\delta_{\mathrm{rot}}$, because $n_{\mathrm{rot}}$ is defined from radial directions measured from that point. If the body origin is the rotation center, $c = 0$.

## 3 The probe curves

**Body twists.**

$$V_t = \big(0,\ R^\top n_{\text{trans}}\big), \qquad V_r = \big(R^\top n_{\text{rot}},\ c \times R^\top n_{\text{rot}}\big). \tag{1}$$

**Probe curves.**

$$a(t) = g\exp(t\hat{V}_t), \qquad b(t) = g\exp(t\hat{V}_r). \tag{2}$$

$V_t$ and $V_r$ are the world-frame normals pulled back into the body frame. The term $c \times R^\top n_{\text{rot}}$ makes the rotation happen about $x_c$ rather than about the body origin.

**Proposition 1** (Closed forms)**.**

$$a(t) = \big(R,\ p + t\, n_{\text{trans}}\big), \tag{3}$$

$$b(t) = \big(R_b(t),\ x_c - R_b(t)\, c\big), \qquad R_b(t) = \operatorname{Exp}\big(t\, n_{\text{rot}}\big)\, R. \tag{4}$$

*At $t = 0$, every body point with world position $x$ has velocity $n_{\text{rot}} \times (x - x_c)$ along $b$, and velocity $n_{\text{trans}}$ along $a$.*

*Proof.* For $a$: $\exp(t\hat{V}_t)$ is the pure translation by $tR^\top n_{\text{trans}}$, and $g$ maps it to $p + tRR^\top n_{\text{trans}}$.

For $b$: let $h = (I, c)$ be the translation by $c$. The adjoint of a pure translation acts as $\operatorname{Ad}_h(\omega, 0) = (\omega, c \times \omega)$, so $V_r = \operatorname{Ad}_h(\omega_b, 0)$ with $\omega_b = R^\top n_{\text{rot}}$. Then $\exp(t\widehat{\operatorname{Ad}_h \xi}) = h\exp(t\hat{\xi})h^{-1}$, and

$$g\,h\,\big(\operatorname{Exp}(t\omega_b), 0\big)\,h^{-1} = (R, x_c)\,\big(\operatorname{Exp}(t\omega_b), 0\big)\,(I, -c) = \big(R\operatorname{Exp}(t\omega_b),\ x_c - R\operatorname{Exp}(t\omega_b)c\big).$$

Finally, $R\operatorname{Exp}(tR^\top n_{\text{rot}})R^\top = \operatorname{Exp}(tn_{\text{rot}})$. The velocity claims follow by differentiating at $t = 0$. □

So you never need a general SE(3) exponential. Equations (3)–(4) are what you implement; the twist form (1) is what you reason with.

*Remark* 1 (The translation probe does not change). Equation (3) is the paper's $q_s + \varepsilon(n_{\text{trans}}, 0)$ exactly. The reformulation only affects the rotation block.

*Remark* 2 (Rotation about the body origin). If $c = 0$, then $V_r = (R^\top n_{\text{rot}}, 0)$ and $b(t) = (\operatorname{Exp}(tn_{\text{rot}})R,\ p)$, so the position does not move. If $c \neq 0$, the position must move in order to keep $x_c$ fixed. The original coordinate probe never moved $p$, so it rotated about the body origin regardless of $c$.

## 4 How the original rotation probe differs

The original probe $r \mapsto r + \varepsilon n_{\text{rot}}$ is $\phi \mapsto \phi + \delta$ with $\delta = 2\pi\varepsilon\, n_{\text{rot}}$. To first order,

$$\operatorname{Exp}(\phi + \delta) = \operatorname{Exp}\big(J_l(\phi)\,\delta + O(\|\delta\|^2)\big)\operatorname{Exp}(\phi), \qquad J_l(\phi) = I + \tfrac{1-\cos\theta}{\theta^2}[\phi]_\times + \tfrac{\theta-\sin\theta}{\theta^3}[\phi]_\times^2.$$

So the original probe turns the body about the world axis $J_l(\phi)n_{\text{rot}}$, not about $n_{\text{rot}}$. Split $n_{\text{rot}}$ into its component along the current axis $u = \phi/\theta$ and the component perpendicular to it. The along-axis component is unchanged. The perpendicular component is rotated about $u$ by $\theta/2$ and scaled by $2\sin(\theta/2)/\theta$. Table 2 gives the worst case, $n_{\text{rot}} \perp u$ (values checked numerically; see Sec. 8).

Near $\theta = \pi$ the original probe can turn the body about an axis orthogonal to the one $n_{\text{rot}}$ asks for. Since RNL penalizes the slope of $S_{\text{rot}}$ in the probed direction, at large rotations the original loss was constraining the wrong directional derivative.

| Current rotation angle $\theta$ | 0° | 45° | 90° | 135° | 180° |
|---|---|---|---|---|---|
| Angle between actual and intended axis | 0° | 22.5° | 45° | 67.5° | 90° |
| Actual turning rate (relative) | 1.000 | 0.974 | 0.900 | 0.784 | 0.637 |

Table 2: Original coordinate probe vs. intended rotation, for $n_{\text{rot}}$ perpendicular to the current rotation axis.

*Remark* 3 (Cheap diagnostic before switching). On a batch of training starts, compute $\cos\angle\big(J_l(\phi)n_{\text{rot}}, n_{\text{rot}}\big)$. If it is close to 1 for most samples (most poses near identity, or $n_{\text{rot}}$ mostly aligned with the current axis), expect a small empirical difference. The change is still a correctness fix, but this tells you how to frame it.

# 5 The loss

With the goal $q_g$ held fixed, define the probe inputs

$$q_{\text{trans}}(\varepsilon) = \big(p + \varepsilon\, n_{\text{trans}},\ r\big), \qquad q_{\text{rot}}(\varepsilon) = \Big(x_c - R_b c,\ \tfrac{1}{2\pi}\operatorname{Log}(R_b)\Big), \quad R_b = \operatorname{Exp}(2\pi\varepsilon\, n_{\text{rot}})\, R. \tag{5}$$

That is, $q_{\text{trans}}(\varepsilon) = a(\varepsilon)$ and $q_{\text{rot}}(\varepsilon) = b(2\pi\varepsilon)$, written in network coordinates. The slope and penalty keep the paper's form:

$$G_c = \frac{S_c\big(q_c(\varepsilon); q_g\big) - S_c(q_s; q_g)}{\varepsilon}, \tag{6}$$

$$L^c_{\text{dir}} = m_c\, \mathbf{1}\big[S^\star_c(q_s) < 1\big]\, \big(\rho_c - \operatorname{clip}(G_c, \rho_c, G_{\max})\big)^2, \tag{7}$$

where $m_{\text{trans}} = 1$ and $m_{\text{rot}}$ is the cut-locus mask of Sec. 6.3. These per-sample terms enter the total objective (the paper's Eq. 8) exactly as before, including causality weighting.

## 5.1 Units and hyperparameters

In Eq. (5) the rotation probe turns by $2\pi\varepsilon$ radians. This choice makes $\varepsilon$, $\rho_{\text{rot}}$ and $G_{\max}$ keep their meaning. At $R = I$ with $c = 0$, $J_l = I$, so $q_{\text{rot}}(\varepsilon)$ is *exactly* the old $r + \varepsilon n_{\text{rot}}$. Away from the identity the two differ only where the old probe was distorted (Table 2). Existing hyperparameters are therefore a sound starting point.

If you prefer the probe size in radians, use $R_b = \operatorname{Exp}(\varepsilon_{\text{rad}} n_{\text{rot}})R$ and rescale the rotation hyperparameters:

$$\varepsilon_{\text{rad}} = 2\pi\varepsilon, \qquad \rho^{\text{rad}}_{\text{rot}} = \rho_{\text{rot}}/2\pi, \qquad G^{\text{rad}}_{\max} = G_{\max}/2\pi.$$

In either convention, $\rho_c$ and $G_{\max}$ are measured in units of $S_c$ per unit $\varepsilon$. If the speed definition changed along with the Eikonal loss, those thresholds may need retuning for that reason alone, independent of this note.

# 6 Numerical details

## 6.1 Probes are data, not part of the graph

The probe poses (5) depend only on $(q_s, n_{\text{trans}}, n_{\text{rot}}, \varepsilon, c)$, not on network parameters. Compute them under `torch.no_grad()`, or once offline and store them next to the training pairs. Then:

- Autodiff never passes through Exp or Log, so their singular points never affect gradients.
- The training-time cost is the same as the original RNL: one extra speed evaluation per block.

### 6.2 A robust logarithm

Log must be accurate near $\theta = 0$ and $\theta = \pi$. The textbook $\theta = \arccos\big((\operatorname{tr} R - 1)/2\big)$ loses precision at both ends. Convert $R$ to a unit quaternion with the largest-diagonal (Shepperd) branch, flip its sign so that $w \geq 0$, and set $\phi = 2\,\operatorname{atan2}(\|v\|, w)\, v/\|v\|$. This always returns $\theta \in [0, \pi]$ and stays accurate everywhere (round-trip error below $10^{-15}$ in the checks of Sec. 8).

### 6.3 The cut locus at $\theta = \pi$

Rotation vectors $\phi$ and $\phi - 2\pi\phi/\theta$ represent the same rotation when $\theta = \pi$. The network input lives in the ball $\|r\| \leq \frac{1}{2}$, with antipodal points on its boundary identified. If $b$ carries the angle past $\pi$, $\operatorname{Log}(R_b)$ jumps from about $+\pi u$ to about $-\pi u$. The probe input then lands on the far side of the ball and $G_{\text{rot}}$ is meaningless.

A legitimate step moves the rotation vector by a small amount. The singular values of $J_l(\phi)^{-1}$ are 1 along the axis and $\frac{\theta}{2\sin(\theta/2)} \leq \frac{\pi}{2}$ across it, so

$$\|\operatorname{Log}(R_b) - \phi\| \leq \tfrac{\pi}{2} \cdot 2\pi\varepsilon + O(\varepsilon^2) = \pi^2\varepsilon + O(\varepsilon^2).$$

A crossing moves it by about $2\pi$. The mask

$$m_{\text{rot}} = \mathbf{1}\big[\|\operatorname{Log}(R_b) - 2\pi r\| < \pi\big] \tag{8}$$

separates the two cases cleanly for any $\varepsilon < 1/\pi$. For Haar-uniform rotations the masked fraction is about $\varepsilon$ (measured: 0.51%, 1.01%, 1.99% at $\varepsilon = 0.005, 0.01, 0.02$).

*Remark* 4 (Recovering masked samples, optional). A forward probe crosses the cut by increasing $\theta$ past $\pi$. The backward probe $b(-2\pi\varepsilon)$ decreases $\theta$, so it generally does not cross. For masked samples you may use the backward difference $G_{\text{rot}} = \big(S(q_s) - S(q_{\text{rot}}(-\varepsilon))\big)/\varepsilon$, which estimates the same slope to $O(\varepsilon)$. If both directions cross (possible only when $n_{\text{rot}}$ is nearly perpendicular to the current axis and $\theta$ is within $O(\varepsilon^2)$ of $\pi$), keep the sample masked.

### 6.4 Planar tasks

SO(2) is abelian, so $J_l = 1$ and $q_{\text{rot}}(\varepsilon) = r + \varepsilon n_{\text{rot}}$ exactly, apart from wrapping at $\theta = \pm\pi$ (which the mask handles). With $c = 0$, the new loss *equals* the old one in 2D, so 2D results should not move. With $c \neq 0$ it differs, through the position compensation in Eq. (4).

# 7 Algorithm

**Algorithm 1** Relative Normal Loss with Lie-group probes (one batch)

**Require:** starts $q_s^{(i)} = (p_i, r_i)$, goals $q_g^{(i)}$, unit normals $n_{\text{trans}}^{(i)}, n_{\text{rot}}^{(i)}$, targets $S_c^\star$, speed function $S_c(\cdot\,;\cdot)$, rotation center $c$, constants $\varepsilon, \rho_c, G_{\max}$

*Precompute, no gradient (can be done offline):*

$R_i \leftarrow \mathrm{Exp}(2\pi r_i), \quad x_{c,i} \leftarrow R_i c + p_i$

$q_{\text{trans}}^{(i)} \leftarrow (p_i + \varepsilon\, n_{\text{trans}}^{(i)},\ r_i)$

$R_i' \leftarrow \mathrm{Exp}(2\pi\varepsilon\, n_{\text{rot}}^{(i)})\, R_i, \quad p_i' \leftarrow x_{c,i} - R_i' c, \quad \phi_i' \leftarrow \mathrm{Log}(R_i')$

$q_{\text{rot}}^{(i)} \leftarrow (p_i',\ \phi_i'/2\pi), \quad m_i \leftarrow \mathbf{1}[\|\phi_i' - 2\pi r_i\| < \pi]$

*Training step:*

Evaluate $S_c$ at $q_s$, $q_{\text{trans}}$ and $q_{\text{rot}}$ in one batched call, goal fixed to $q_g$

$G_{\text{trans}} \leftarrow \big(S_{\text{trans}}(q_{\text{trans}}) - S_{\text{trans}}(q_s)\big)/\varepsilon, \quad G_{\text{rot}} \leftarrow \big(S_{\text{rot}}(q_{\text{rot}}) - S_{\text{rot}}(q_s)\big)/\varepsilon$

$L_i^{\text{trans}} \leftarrow \mathbf{1}[S_{\text{trans}}^\star < 1]\,(\rho_{\text{trans}} - \mathrm{clip}(G_{\text{trans}}, \rho_{\text{trans}}, G_{\max}))^2$

$L_i^{\text{rot}} \leftarrow m_i\, \mathbf{1}[S_{\text{rot}}^\star < 1]\,(\rho_{\text{rot}} - \mathrm{clip}(G_{\text{rot}}, \rho_{\text{rot}}, G_{\max}))^2$

**return** per-sample $L^{\text{trans}}, L^{\text{rot}}$ ▷ weighted into Eq. 8 as before

If the Eikonal term already computes $S_c(q_s; q_g)$ in the same step, reuse it and evaluate only the two probe inputs.

# 8 Verification checklist

The reference code in Appendix A passes the following checks (float64, 4000 random poses, including angles within $10^{-7}$ of 0 and $\pi$). They are worth rerunning after porting the code into your training repository.

| Check | What it confirms | Result |
|---|---|---|
| $\mathrm{Log}(\mathrm{Exp}(\phi)) = \phi$ | robust logarithm | max error $8.9 \times 10^{-16}$ |
| $R_b c + p_b = x_c$ | rotation center stays fixed | max drift $5.5 \times 10^{-16}$ |
| Closed form vs. $g\exp(t\hat{V})$ via `matrix_exp` | Proposition 1 | max error $1.3 \times 10^{-15}$ |
| Point velocity $= n_{\text{rot}} \times (x - x_c)$ | probe turns about the right axis | error $1.9 \times 10^{-6}$ (FD, $h = 10^{-6}$) |
| New = old probe at $R = I$, $c = 0$ | hyperparameters transfer | $5.2 \times 10^{-18}$ |
| Misalignment of old probe | Table 2 | matches $\theta/2$, $2\sin(\theta/2)/\theta$ |
| Crossing probes masked, others kept | cut-locus mask | 100% / 0% |
| Loss backpropagates to parameters | end-to-end wiring | finite loss, nonzero gradient |

# A Reference implementation (PyTorch)

The `speed_fn` argument is your own speed function. It must return $(S_{\text{trans}}, S_{\text{rot}})$, each of shape $(N,)$, and be differentiable with respect to the network parameters (for gradient-based speeds, use `create_graph=True`). For example, the paper's definition is $S_c = \|\nabla_{q_c} T\|^{-1}$; any replacement works the same way.

```python
"""Relative Normal Loss with Lie-group probe curves (reference implementation).

Pose g = (R, p) in SE(3). Network input q = (p, r) with r = Log(R) / (2*pi).
Only the probe curves change; the speed function S_c is supplied by the caller.
"""
import math
import torch

TWO_PI = 2.0 * math.pi


def hat(w):
    """(..., 3) -> (..., 3, 3) skew-symmetric matrix [w]_x."""
    wx, wy, wz = w.unbind(-1)
    O = torch.zeros_like(wx)
    return torch.stack([torch.stack([O, -wz, wy], -1),
                        torch.stack([wz, O, -wx], -1),
                        torch.stack([-wy, wx, O], -1)], -2)


def so3_exp(w):
    """Rodrigues formula, stable for small angles. w: (..., 3) rotation vector."""
    th2 = (w * w).sum(-1, keepdim=True)[..., None]          # (..., 1, 1)
    th = th2.clamp_min(1e-24).sqrt()
    small = th2 < 1e-8
    A = torch.where(small, 1 - th2 / 6, torch.sin(th) / th)
    B = torch.where(small, 0.5 - th2 / 24, (1 - torch.cos(th)) / th2.clamp_min(1e-24))
    K = hat(w)
    I = torch.eye(3, dtype=w.dtype, device=w.device).expand(K.shape)
    return I + A * K + B * (K @ K)


def so3_log(R):
    """Rotation vector with angle in [0, pi]. Robust at 0 and pi (via quaternion)."""
    m00, m01, m02 = R[..., 0, 0], R[..., 0, 1], R[..., 0, 2]
    m10, m11, m12 = R[..., 1, 0], R[..., 1, 1], R[..., 1, 2]
    m20, m21, m22 = R[..., 2, 0], R[..., 2, 1], R[..., 2, 2]
    t = m00 + m11 + m22
    # four candidate (unnormalized) quaternions (w, x, y, z); pick the best-conditioned
    cands = torch.stack([
        torch.stack([1 + t, m21 - m12, m02 - m20, m10 - m01], -1),
        torch.stack([m21 - m12, 1 + 2 * m00 - t, m01 + m10, m02 + m20], -1),
        torch.stack([m02 - m20, m01 + m10, 1 + 2 * m11 - t, m12 + m21], -1),
        torch.stack([m10 - m01, m02 + m20, m12 + m21, 1 + 2 * m22 - t], -1),
    ], -2)                                                  # (..., 4, 4)
    diag = torch.stack([t, m00, m11, m22], -1)
    idx = diag.argmax(-1)
    q = torch.gather(cands, -2, idx[..., None, None].expand(*idx.shape, 1, 4)).squeeze(-2)
    q = q / q.norm(dim=-1, keepdim=True)
    q = torch.where(q[..., :1] < 0, -q, q)                  # w >= 0  =>  angle <= pi
    v = q[..., 1:]
```

```
    s = v.norm(dim=-1, keepdim=True)
    ang = 2 * torch.atan2(s, q[..., :1])
    scale = torch.where(s < 1e-12, 2 / q[..., :1].clamp_min(1e-12), ang / s.clamp_min(1e-12))
    return scale * v


def body_twists(R, n_trans, n_rot, c_body=None):
    """Body twists (omega, v) for the two probes, returned as (..., 6) = [omega, v].
    n_trans, n_rot: unit world-frame vectors. c_body: rotation centre, body frame.
    (Not needed by make_probes, which uses the closed forms; kept for reference/tests.)"""
    Rt = R.transpose(-1, -2)
    w_b = (Rt @ n_rot[..., None])[..., 0]
    V_t = torch.cat([torch.zeros_like(n_trans), (Rt @ n_trans[..., None])[..., 0]], -1)
    if c_body is None:
        v_r = torch.zeros_like(w_b)
    else:
        v_r = torch.cross(c_body.expand_as(w_b), w_b, dim=-1)
    V_r = torch.cat([w_b, v_r], -1)
    return V_t, V_r


@torch.no_grad()
def make_probes(q_s, n_trans, n_rot, eps, c_body=None, cut_thresh=math.pi):
    """Probe inputs a(eps), b(eps) for a batch of start configurations.

    q_s:     (B, 6) network inputs (p, r), r = Log(R) / (2 pi)
    n_trans: (B, 3) unit world-frame translation normal
    n_rot:   (B, 3) unit world-frame rotation axis (body radial x obstacle radial)
    eps:     probe size in normalized units; the rotation probe turns by 2*pi*eps rad
    c_body:  (3,) or (B, 3) rotation centre in body frame (None = body origin)
    Returns q_trans_probe, q_rot_probe (B, 6) and valid_rot (B,) bool mask.
    """
    p, r = q_s[:, :3], q_s[:, 3:]
    w_s = TWO_PI * r
    R = so3_exp(w_s)

    # translation probe a(eps) = g exp(eps * V_t) = (R, p + eps n_trans)
    q_t = torch.cat([p + eps * n_trans, r], -1)

    # rotation probe b(eps) = g exp(2 pi eps * V_r): rotate about world axis n_rot, centre fixed
    R_new = so3_exp(TWO_PI * eps * n_rot) @ R
    if c_body is None:
        p_new = p
    else:
        c = c_body.expand_as(p)
        x_c = (R @ c[..., None])[..., 0] + p     # world position of the centre
        p_new = x_c - (R_new @ c[..., None])[..., 0]
    w_new = so3_log(R_new)
    q_r = torch.cat([p_new, w_new / TWO_PI], -1)

    # cut locus: a legal step moves the rotation vector by <= (pi/2) * 2*pi*eps;
    # crossing angle pi flips it by ~2*pi. Mask those probes.
    valid = (w_new - w_s).norm(dim=-1) < cut_thresh
    return q_t, q_r, valid


def relative_normal_loss(speed_fn, q_s, q_g, q_t, q_r, valid_rot, S_star, eps, rho, G_max):
    """Per-sample RNL terms (Eq. 7) for c in {trans, rot}.

```

```
    speed_fn(q, q_g) -> (S_trans, S_rot), each (N,), differentiable w.r.t. the
        network parameters (use create_graph=True inside). Any speed definition works.
    S_star: dict {'trans': (B,), 'rot': (B,)} geometry targets at q_s.
    rho, G_max: dicts of per-block thresholds.
    Returns dict of (B,) losses; apply causality weights / averaging outside.
    """
    B = q_s.shape[0]
    q_all = torch.cat([q_s, q_t, q_r], 0)
    S_tr, S_ro = speed_fn(q_all, q_g.repeat(3, 1))            # goal held fixed
    S0 = {'trans': S_tr[:B], 'rot': S_ro[:B]}
    S1 = {'trans': S_tr[B:2 * B], 'rot': S_ro[2 * B:]}
    out = {}
    for c in ('trans', 'rot'):
        G = (S1[c] - S0[c]) / eps
        L = (rho[c] - torch.clamp(G, min=rho[c], max=G_max[c])) ** 2
        active = (S_star[c] < 1).to(L.dtype)
        if c == 'rot':
            active = active * valid_rot.to(L.dtype)
        out[c] = active * L
    return out
```

Example wiring:

```
# offline / in the data loader
q_t, q_r, valid = make_probes(q_s, n_trans, n_rot, eps, c_body=c)   # c=None if centre is body
    origin

# training step
L = relative_normal_loss(speed_fn, q_s, q_g, q_t, q_r, valid, S_star, eps,
                         rho={'trans': rho_t, 'rot': rho_r},
                         G_max={'trans': G_max, 'rot': G_max})
loss_dir = (w_causal * (L['trans'] + L['rot'])).mean()
```